CernVM-FS *Filebundles* for low-latency software distribution in interactive use cases

Valentin Volkl[1,*], *Georgios* Christodoulis[1], and *Jakob* Blomer[1]

[1]CERN, 1211 Geneva 23, Switzerland

Abstract. CernVM-FS is a read-only, on-demand file system used to distribute scientific software to the Worldwide LHC Computing Grid and many communities beyond high-energy physics. Its lazy, per-file fetching is highly efficient for batch computing, but it adds noticeable latency to interactive sessions: starting an application that requires access to hundreds of small files on a cold cache is triggering a chain of serial downloading operations to the network. We present *filebundle* prefetching, a mechanism that exploits the predictable file-access pattern at application startup. A small bundle specification called *filebundle*, published next to a trigger file, declares the set of files needed immediately after the trigger is opened. When a client opens the trigger, CernVM-FS prefetches the listed dependencies concurrently using a pool of worker threads, replacing a long serial chain of requests with a short parallel prefetch phase. On a ROOT startup benchmark, *filebundles* bring performance into the local-disk range and keep startup time nearly flat as network latency grows, while the on-demand baseline degrades approximately linearly. *Filebundles* are an opt-in feature in CernVM-FS 2.14.

1 Introduction

CernVM-FS [1] is a global, read-only file system for distributing scientific software. It is a core component of the Worldwide LHC Computing Grid (WLCG) and is also used by communities beyond high-energy physics, including Euclid, LIGO, LSST, SKA, and EESSI.

CernVM-FS uses a distributed architecture (Fig. 1), where software is originally published in a Stratum0 servere - is considered the source of truth, in the form of repositories. The contents of Stratum0 are then cloned in a series of geographically scattered mirror servers (Stratum1's), arranged to facilitate the different user sites, in order to mitigate the traffic of Stratum0. Users can access the contents of a repository either directly from a Stratum (0 or 1) (not encouraged) or via a series of forward proxies that site administrators configure, for better load balancing and caching. WLCG has about fifteen Stratum 1 mirror servers in Europe, the Americas, Australia, and Asia, managing about 3.3×10^{10} files and 2 PB of data and more than 8000 container images.

Data (repository contents) on CernVM-FS are stored in a content addressable format. During publication files are chunked, hashed and compressed. A filename is associated to its hash (or hashes in the case of chunking) through dedicated metadata dictionaries called

*e-mail: valentin.volkl@cern.ch

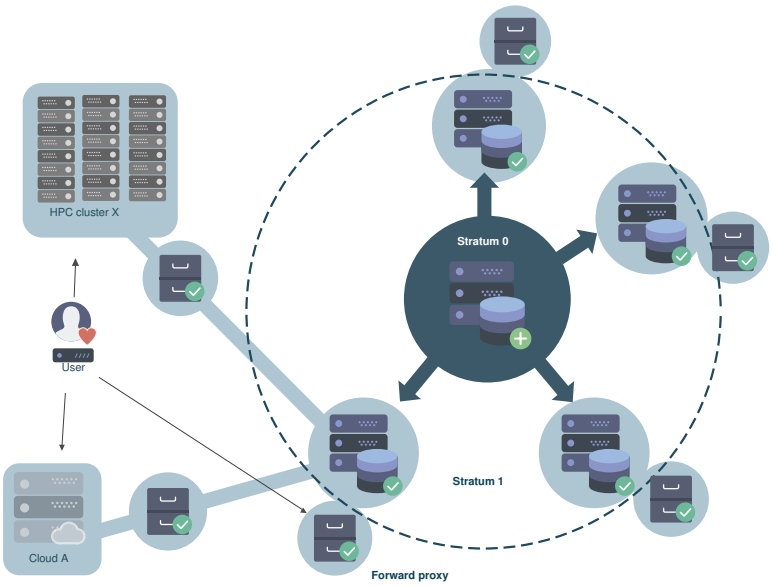


Figure 1. The CernVM-FS cache hierarchy: a writable Stratum 0 origin, globally distributed Stratum 1 mirrors, site-level Squid proxies, and per-client disk and kernel page caches. The *filebundle* mechanism operate on the same hierarchy, only changing proxy's workload, creating short bursts of download requests on the first access of a trigger file.

catalogs. Catalogs can also be nested in order to keep catalog operations manageable for large repositories.

From the users perspective, repositories can be accessed via a global namespace, by default under */cvmfs* in their machine. This is done by the CernVM-FS client, which is a FUSE based filesystem. Typically, there is some disk allocated space in the user's machine that works as a local cache. CernVM-FS is performing cache cleanup using an LRU policy; once the quota limit is reached, the cache manager will elect some files for eviction prioritizing those least recently used. The quota limit is a soft limit, and cleanup efforts will continue until the cache size reaches a specific target (by default half the size of the quota limit).

For simple listing operations, the client will fetch the associated catalogs which contain all the information necessary. When an *open()* is issued from the user, the client will lookup for the correct hash, and will issue a *fetch()*. During fetching, the client will download the compressed hash, will decompress it and store it in the local cache, available for reading. On a *read()* the client will make the contents of the files available to the user immediately, or will trigger more *fetch()* operations in case the file is chunked, or the hash has already been evicted from the cache. According to this mechanism, on first access the cache is empty (cold cache) and the contents of the repository become available after they are needed. Downloading, decompression, and caching are transparent to the user.

This lazy, content-addressed model is extremely effective for distributed batch computing, where the benefits of a shared software distribution system dominate any per-file overhead. In interactive sessions, however, the same on-demand behaviour can make application startup noticeably slower than from local disk. This paper presents filebundles, a mechanism that reduces that startup latency by prefetching the files an application needs immediately after launch.

2 Motivation

CernVM-FS was designed from the beginning with great emphasis on performance. Its architecture portends some weak points that have thoroughly been investigated so far. The first and major one is the overhead added by the network. This series of mirror and proxy servers are aiming to improve data locality, opting to minimize its non negligible impact. The second factor is the overhead added by FUSE which is a performance penalty that can not be avoided. FUSE allows the filesystem to be designed in user space, which is a strong requirement by the community (since a kernel module approach is not considered a viable option).

At the CernVM Users Workshop in 2022, Chris Burr (LHCb) reported that interactive CernVM-FS use can be roughly three to five times slower than local disk [2]. This overhead is usually acceptable for distributed batch workloads, but is noticeable when starting an interactive analysis environment.

The behavior is easy to reproduce on a controlled testbed. Installing ROOT from a release tarball on local disk, importing ROOT in Python takes about 2.47 s. Using the same software stack from */cvmfs* with a cold cache, the time increases to about 4.69 s. In this setup the network path is very short, so the slowdown is not primarily a bandwidth problem. It confirms that a significant part of the delay comes from the way CernVM-FS serves many files on demand.

2.1 Cold cache performance analysis

When an application such as ROOT starts, it opens many dependent files one after another. On a cold cache, for each file CernVM-FS handles a FUSE upcall, issues an HTTP request, waits for the round trip, downloads the object, decompresses it , stores it in cache and returns the data. This per-file overhead is paid for every dependent file, and is effectively serialized among the dependencies. The per-file amount of work is unremarkable, but the latency across the multiple serialized requests adds up, and becomes noticeable in interactive use.

The idea behind filebundles is that we can take advantage of the fact that the dependencies of an application are or can be known at the time of the publication of the repository, and all dependencies can be fetched simultaneously after the main (trigger) file is opened. The *filebundle* mechanism operates within the existing hierarchy of CernVM-FS (Fig. 1). There are a couple additional points of complexity that come with the feature conceptually. From the software maintainers standpoint, there is the responsibility to provide an up to date dependency list during publishing new versions of their software. For the site infrastructure administrators, there is some change in the proxy access patterns to be anticipated (since on a cold cache, massive parallel download requests will be triggered from the client).

3 Filebundles

Filebundles use prior knowledge of the application's source file dependencies during startup. For applications such as ROOT [3] or Python, there is a main file (*trigger*) whose access is signaling the application's startup. The series of files that are accessed immediately after the trigger file is opened are often predictable. A bundle records these dependencies allowing the client to fetch them eagerly in parallel before they are needed from the application.

A bundle is declared by a specification file published next to its trigger file in the same directory, named *.cvmfsbundle-<filename>*, where *<filename>* is the trigger. The specification is a small, versioned JSON document holding minimal metadata and a flat list of dependency paths, expressed as absolute paths from the repository root. A representative ROOT bundle, keyed on the Python package *__init__.py*, contains roughly 250 paths:

```
#%CVMFS_BUNDLE version=1 encoding=UTF-8
{
  "name": "CVMFS_BUNDLE",
  "version": "1.0.0",
  "encoding": "UTF-8",
  "dependencies": [
    "/root/lib/libCore.so",
    "/root/lib/libRIO.so",
    "/root/lib/libcppyy.so",
    "/root/lib/ROOT/_application.py",
    ... (~250 paths total)
  ]
}
```

When a client receives an *open()* on the trigger file, CernVM-FS detects the matching *.cvmfsbundle* specification and begins prefetching the listed dependencies concurrently, using a pool of worker threads, while the application continues to run. Once fetched, the files (which from the clients point of view is a series of compressed hashes) are getting decompressed (also in parallel) and stored in the local disk. In effect, a long serial chain of dependent requests is replaced by a much shorter parallel prefetch phase that warms the local cache ahead of the application's subsequent *open()* calls.

The feature is disabled by default and can be enabled per mountpoint with the configuration parameter *CVMFS_PREFETCH_FILEBUNDLES=yes* [4]. The setting is read at mount time; mounts for which it is disabled do not start bundle prefetching. The prefetch-pool size is controlled by *CVMFS_BUNDLE_POOL_SIZE*.

3.1 Publishing a filebundle

In order to create *.cvmfsbundle-<filename>*, there is a straightforward series of steps. First, choose a suitable trigger file, for example a Python *__init__.py* or a launcher binary such as *root.exe*. Second, determine which files are typically read after that trigger; this can be done by hand or with tracing tools such as *strace* or the built-in *CVMFS_TRACEFILE* mechanism. Third, publish the resulting *.cvmfsbundle-<filename>* specification next to the trigger file, just like any other file in the repository. Removing the specification (and republishing) removes the bundle, after which the trigger behaves like an ordinary file.

4 Results

Unless noted otherwise, the benchmarks use ROOT 6.36.12 in a test CernVM-FS repository served from *localhost*. The client ran on an AMD EPYC 7302 system (2×16 cores, 64 threads at ~3.0 GHz, 251 GB RAM), with AlmaLinux 10.1 and kernel 6.12. Using *localhost* removes external network effects, exposing the per-file software overhead.

4.1 Startup performance

Figure 2 compares ROOT startup times on local disk, on CernVM-FS with a cold cache and no *filebundles*, and on CernVM-FS with *filebundles* enabled. In this benchmark, *filebundles* bring CernVM-FS startup time to the local-disk range. Some measurements are marginally faster than local disk, consistent with overlap between downloading and decompression in the prefetch path.

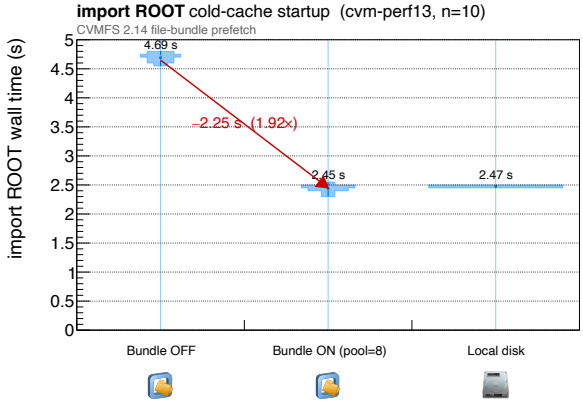


Figure 2. A violin plot of the distribution of startup times over ten repetitions of importing ROOT. With *filebundles* enabled (*CVMFS_PREFETCH_FILEBUNDLES=ON*), CernVM-FS startup matches local-disk performance, closing the gap to the on-demand cold-cache baseline.

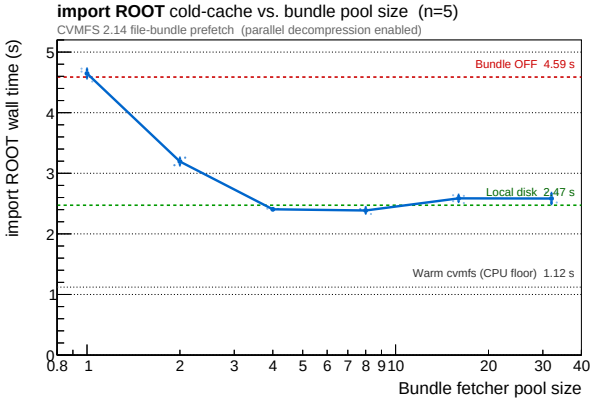


Figure 3. ROOT startup time as a function of the prefetch thread-pool size (*CVMFS_BUNDLE_POOL_SIZE*). Performance saturates around eight worker threads, the default.

4.2 Prefetch thread pool

An efficient prefetching mechanism should ensure that the files inside the bundle are fetched as rapidly as possible to be available on potential access. During our primitive experiments, we realized that a single-threaded fetching mechanism inside the client is getting saturated and the network is being underutilized (every download is followed by a decompression from the same worker thread). To overcome this bottleneck, we implemented prefetching with a pool of worker threads whose size is set by *CVMFS_BUNDLE_POOL_SIZE*. As shown in Fig. 3, startup performance improves as workers are added and saturates around eight threads, which is also the default. Most of the benefit is therefore obtained with a moderate amount of concurrency and without aggressive tuning.

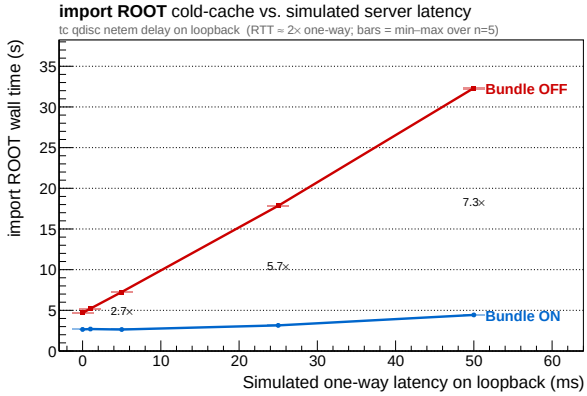


Figure 4. ROOT startup time as a function of injected round-trip latency. The on-demand baseline degrades roughly linearly, while *filebundles* keep startup nearly flat by overlapping requests.

4.3 Network latency

For the experiments of section 4.2 shown in Fig. 3, the repository was served from localhost, which minimizes the effect of network latencies that appear on more realistic scenarios. To comprehend the impact of network latency to the performance of *filebundles*, we injected loopback round-trip latency with *tc qdisc netem*. Without *filebundles*, startup time grows approximately linearly with round-trip time because the latency of the sequential dependent small-file fetches grows cumulatively. With *filebundles*, startup time remains nearly flat over a substantial range (Fig. 4): concurrent prefetch pays the round-trip penalty for a small number of batches rather than for each file. This is an idealized loopback experiment.

4.4 Caveats

Filebundles do not reduce the total amount of data transferred; they compress the startup traffic into a short, intense burst. In the benchmark, requests peak at about 1480 per second with *filebundles*, versus about 680 per second without them (Fig. 5). The feature should therefore be used selectively, targeting interactive systems such as *lxplus*, where startup latency matters most. Over-prefetching should be avoided, and any rollout should be coordinated with proxy and site administrators so that the request bursts do not overload the sites' proxies.

5 Outlook

CernVM-FS since version 2.14 takes advantage of the FUSE passthrough mechanism introduced in the linux kernel version 6.9 (available in EL10 and on), and can hand over to the kernel a descriptor for the cached backing file during *open()*, so subsequent reads bypass FUSE. This feature works only for unchunked files and eliminates the overhead imposed by the FUSE call hierarchy. Its potential to improve the startup time of application combined with *filebundles* still needs to be benchmarked. Planned work for CernVM-FS 2.15 includes tools for creating and updating bundles, support for file chunks in bundle specifications, and integration with build tools. Packfiles, which would combine small backend objects to reduce request counts, are also under consideration; backward compatibility is the principal obstacle.

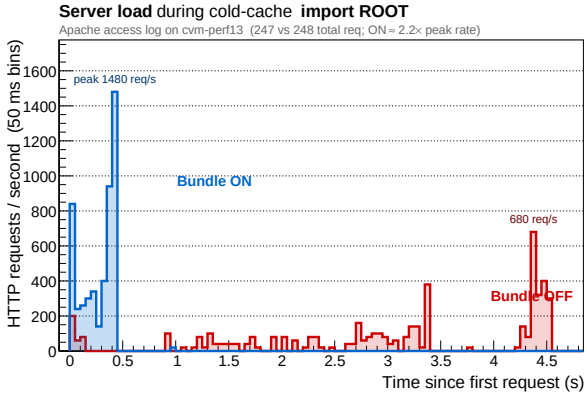


Figure 5. Request rate during startup. *Filebundles* concentrate the same total traffic into a shorter, more intense burst (peak ∼1480 req/s versus ∼680 req/s without bundles).

6 Conclusions

Filebundles reduce application-startup latency when a known set of files is needed immediately after a trigger is opened. They require the repository maintainer to trace startup behaviour and publish a *.cvmfsbundle* specification next to the trigger. In the ROOT benchmark, they reduce cold-start time to the local-disk range and make startup less sensitive to added round-trip latency. *Filebundles* are an opt-in CernVM-FS 2.14 feature, enabled per mount with *CVMFS_PREFETCH_FILEBUNDLES*. They are best suited to interactive systems, where the prefetch burst can be managed.

Acknowledgements. We thank Chris Burr (LHCb) for the valuable input that provided a measurable reference for the potential of improving applications' startup on interactive workloads.

Disclosure: AI-based tools were used to assist with drafting and formatting this manuscript; all content was reviewed and verified by the authors.

References

[1] J. Blomer, C. Aguado-Sánchez, P. Buncic, A. Harutyunyan, Distributing LHC application software and conditions databases using the CernVM file system, J. Phys.: Conf. Ser. **331**, 042003 (2011). https://doi.org/10.1088/1742-6596/331/4/042003

[2] C. Burr, *Client performance*, CernVM Users Workshop (2022). https://indico.cern.ch/event/1079490/contributions/4939521/attachments/2506388/4306645/2022-09-12_CVMFS-Workshop_LHCb.pdf

[3] R. Brun, F. Rademakers, ROOT—an object oriented data analysis framework, Nucl. Instrum. Methods Phys. Res. A **389**, 81–86 (1997). https://doi.org/10.1016/S0168-9002(97)00048-X

[4] CernVM-FS documentation. https://cvmfs.readthedocs.io/